

STUDENT WORKBOOK

PLC Modbus RTU Communication

Micro820 to Automation Direct GS10/GS11 VFD

This workbook teaches you how to set up a Micro820 PLC to control an Automation Direct GS10 or GS11 variable frequency drive (VFD) over Modbus RTU serial communication. By the end of these labs, you will be able to start, stop, change speed, and read status from a VFD using ladder logic in Connected Components Workbench (CCW).

	Details
Course	Industrial Controls / PLC Programming
Equipment	Micro820 PLC + GS10 or GS11 VFD + 3-phase motor
Software	Connected Components Workbench (CCW) v12+
Prerequisites	Basic ladder logic, electrical safety, VFD keypad operation
Total Labs	7 labs (approximately 3-4 hours total)
Safety Gear	Safety glasses, closed-toe shoes, no loose clothing near rotating equipment

Student Name: _____

Date: _____

Instructor: _____

Lab Station #: _____

SAFETY FIRST

Before you touch any equipment in this lab, read and understand these safety rules. Your instructor must verify you have read this page before you begin.

■ **SAFETY: NEVER** connect or disconnect wiring while the VFD is powered on. Always verify zero voltage with a meter before touching terminals.

■ **SAFETY: ALWAYS** disconnect the motor from its mechanical load during initial commissioning. An unexpected start at full speed can cause injury or equipment damage.

■ **SAFETY:** The VFD keypad STOP button must remain active (P00.31 = 02) during all labs. This is your emergency backup if the PLC program has a bug.

■ **SAFETY:** RS-485 communication wiring carries low-voltage signals, but the VFD power terminals carry lethal voltage. Stay away from the power terminal side of the drive.

I have read and understand all safety warnings above. I will follow all shop safety rules and my instructor's directions.

Student Signature: _____ Date: _____

Instructor Initials: _____

TABLE OF CONTENTS

Lab 1: VFD Parameter Setup (Drive-Side Configuration)

Lab 2: Wiring the RS-485 Connection

Lab 3: CCW Project Setup and Serial Port Configuration

Lab 4: Building the Scan Timer (Your First Rung)

Lab 5: Writing to the Drive (MSG_MODBUS Write)

Lab 6: Reading from the Drive (MSG_MODBUS Read)

Lab 7: Putting It All Together — Run/Stop/Speed Control

Reference: Complete Tag List and Control Word Values

LEARNING OBJECTIVES

1. Identify the communication parameters on the GS10/GS11 VFD keypad.
2. Set the drive to accept Run/Stop and Speed commands from RS-485.
3. Configure the baud rate, parity, and protocol to match the PLC.

BACKGROUND

Before the PLC can talk to the drive, you have to tell the drive two things: (1) where to get its run/stop and speed commands from (answer: the RS-485 serial port), and (2) what communication settings to use (baud rate, parity, etc). Think of it like tuning two walkie-talkies to the same channel.

PROCEDURE

Step 1: Power Up the Drive (No Motor Connected)

Apply power to the GS10/GS11. The keypad should light up with the default display.

Make sure no motor is connected to the output terminals (U/T1, V/T2, W/T3).

Step 2: Set the Operation Source to RS-485

Using the keypad, navigate to **P00.21** and set it to **02**.

This tells the drive: "Get your Run/Stop commands from the RS-485 port."

Navigate to **P00.20** and set it to **1**.

This tells the drive: "Get your speed (frequency) command from RS-485."

Step 3: Set Communication Parameters

Navigate to **P09.00** (Communication Address). Set it to **1**.

Navigate to **P09.01** (Baud Rate). Set it to **19.2** (19200 baud).

Navigate to **P09.04** (Protocol). Set it to **14**.

■ **CAUTION:** P09.04 = 14 means: 8 data bits, Even parity, 1 stop bit, RTU mode. This is the #1 most common mistake in Modbus setups. If you set this wrong, every byte the PLC sends will be garbled. The default on a new GS10 is 13 (8-N-2) which will NOT work with the Micro820's default settings.

Here is the complete table of P09.04 RTU settings. Circle the one you just set:

P09.04	Data Bits	Parity	Stop Bits	Mode
12	8	None	1	RTU

P09.04	Data Bits	Parity	Stop Bits	Mode
13 (factory default)	8	None	2	RTU
14 (use this one)	8	Even	1	RTU
15	8	Odd	1	RTU
16	8	Even	2	RTU
17	8	Odd	2	RTU

✓ **CHECKPOINT — Verify Before Continuing**

What value did you set P00.21 to? _____ What does it mean? _____

What value did you set P09.04 to? _____ What parity does that use? _____

What is the communication address of your drive (P09.00)? _____

NOTES:

LAB 2

Wiring the RS-485 Connection

Estimated Time: 15 minutes

LEARNING OBJECTIVES

1. Identify the RS-485 pins on the GS10/GS11 RJ45 connector.
2. Identify the RS-485 terminals on the Micro820 PLC.
3. Wire a working RS-485 connection between the PLC and the drive.

■ **SAFETY:** Verify BOTH the PLC and VFD are powered OFF before making any wiring connections.

BACKGROUND

RS-485 is a two-wire differential signal. It uses an A line (Data+) and a B line (Data-). The voltage difference between A and B carries the data. You also need a signal ground (SGND) connected between the two devices. The GS10 uses an RJ45 jack, and the Micro820 uses screw terminals.

GS10/GS11 RJ45 PINOUT

RJ45 Pin	Signal	Wire To
3	SGND (Signal Ground)	Micro820 Pin 5 (GND)
4	SG- (Data B / negative)	Micro820 Pin 2 (TX-/RX-)
5	SG+ (Data A / positive)	Micro820 Pin 1 (TX+/RX+)
1, 2, 6, 7	Reserved	DO NOT CONNECT
8	+10VS (+10V supply)	DO NOT CONNECT

PROCEDURE

Step 1: Prepare the Cable

You need a cable with an RJ45 plug on one end (for the GS10) and bare wires on the other end (for the Micro820 screw terminals). Use shielded twisted pair cable.

Step 2: Connect the Wires

RJ45 Pin 5 (SG+) -----> Micro820 screw terminal Pin 1 (TX+/RX+)

RJ45 Pin 4 (SG-) -----> Micro820 screw terminal Pin 2 (TX-/RX-)

RJ45 Pin 3 (SGND) -----> Micro820 screw terminal Pin 5 (GND)

Step 3: Verify with a Multimeter (Continuity Check)

Set your meter to continuity/beep mode.

Touch one probe to the RJ45 Pin 5 wire, the other to Micro820 Pin 1. Should beep.

Repeat for Pin 4 to Pin 2, and Pin 3 to Pin 5.

✓ CHECKPOINT — Verify Before Continuing

Which GS10 RJ45 pin is Data+ (SG+)? _____

Which Micro820 terminal did you connect it to? _____

Did your continuity check pass on all 3 connections? ☐ Yes ☐ No

NOTES:

LEARNING OBJECTIVES

1. Create a new CCW project for the Micro820.
2. Configure the serial port as Modbus RTU Master.
3. Create all the tags (variables) needed for the program.
4. Download the configuration to the PLC.

PROCEDURE

Step 1: Create New Project

Open CCW. Click **File > New**.

Select your Micro820 controller model (e.g., 2080-LC20-20QWB).

Name the project **GS10_Modbus_Lab**. Click **Create**.

Step 2: Configure the Serial Port

In the Project Organizer (left panel), click the **Micro820** icon.

Click the **Serial Port** tab.

Set **Driver**: Modbus RTU

Set **Modbus Role**: Modbus RTU Master

Set **Physical Layer**: RS-485

Set **Baud Rate**: 19200 | **Parity**: Even | **Stop Bits**: 1

Step 3: Add a Ladder Program

Right-click **Programs > Add > New LD : Ladder Diagram**.

Right-click the new program and rename it to **ModbusControl**.

Step 4: Create Your Tags

Double-click **Global Variables**. Create every tag in the table below.

For each row: click the blank row, type the Name, select the Data Type, set the Initial Value.

Tag Name	Data Type	Initial Value	What It Does
ControlWord	INT	0	Run/Stop/Direction command sent to drive
FreqCommand	INT	0	Speed command (0.01 Hz units, 6000=60Hz)
StatusData	INT[8]	all 0	Status values read back from drive
DriveError	INT	0	Fault code from drive

Tag Name	Data Type	Initial Value	What It Does
ActualFreq	INT	0	Actual motor frequency
RunCommand	BOOL	FALSE	Run button
StopCommand	BOOL	FALSE	Stop button
SpeedSetpoint	INT	3000	Desired speed (3000 = 30 Hz)
ScanTimerDone	BOOL	FALSE	Timer heartbeat pulse
CommOK	BOOL	FALSE	Communication healthy flag
CommFault	BOOL	FALSE	Communication error flag

✓ **CHECKPOINT — Verify Before Continuing**

What baud rate did you set in CCW? _____ Does it match the VFD (P09.01)? [] Yes [] No

What parity did you set in CCW? _____ Does it match P09.04 = 14? [] Yes [] No

How many tags did you create? _____

Step 5: Build and Download

Right-click the Micro820 controller > **Build**. Fix any errors (0 errors required).

Connect to PLC via USB. Click **Download**.

LAB 4

Building the Scan Timer

Estimated Time: 15 minutes

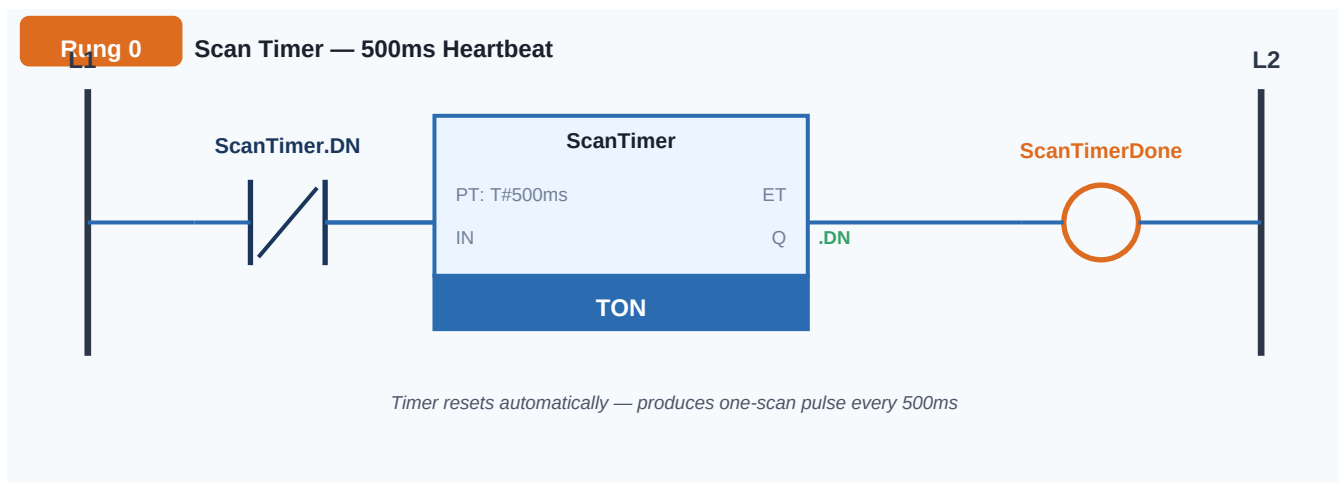
LEARNING OBJECTIVES

1. Understand why Modbus messages need a timed trigger (not continuous).
2. Build a self-resetting TON timer in ladder logic.
3. Verify the timer produces a pulse every 500ms.

BACKGROUND

The MSG_MODBUS instruction sends one Modbus message each time it is triggered. If you leave the trigger on all the time, it will flood the serial bus and cause errors. Instead, we use a timer to send messages at a controlled rate — like a heartbeat. Every 500 milliseconds (half a second), the timer fires a one-scan pulse that starts our Modbus message chain.

THE RUNG



PROCEDURE

Step 1: Open Your Ladder Program

Double-click **ModbusControl** in the Project Organizer.

Step 2: Drag a TON Timer onto Rung 0

Click the **Toolbox** tab (lower right). Expand **Ladder**.

Find **TON** (Timer On Delay). Drag it onto the first empty rung.

In the dialog, name the instance **ScanTimer**. Click OK.

Step 3: Set the Preset Time

Click on the **PT** input of the TON block. Type **T#500ms**.

This means the timer will count up for 500 milliseconds before firing.

Step 4: Add the Self-Reset Contact

From the Toolbox, drag a **Normally Closed Contact** (the one with the slash) to the **LEFT** of the TON block.

Click the contact label. Assign it to **ScanTimer.DN**.

Why does this work? When the timer finishes (DN=TRUE), the NC contact opens, which removes the input to the timer, which resets it, which closes the contact again. It is a self-resetting loop.

Step 5: Add the Output Coil

Drag a **Coil** to the RIGHT side of the rung. Assign it to **ScanTimerDone**.

This BOOL tag will pulse TRUE for exactly one PLC scan every 500ms.

INSTRUCTOR TIP: Have students go online and watch the ET (Elapsed Time) value count up to 500ms, then reset to 0. This helps them understand the self-resetting pattern.

✓ CHECKPOINT — Verify Before Continuing

What is the preset time of your timer? _____

What type of contact resets the timer? [] Normally Open [] Normally Closed

How often does ScanTimerDone pulse TRUE? Every _____ ms

REVIEW QUESTIONS

1. Why can't you just leave the MSG_MODBUS enable input on all the time?

Answer: _____

2. If you wanted to poll the drive faster (every 250ms), what would you change?

Answer: _____

3. What does the .DN output of a TON timer mean?

Answer: _____

LAB 5

Writing to the Drive (MSG_MODBUS)

Estimated Time: 30 minutes

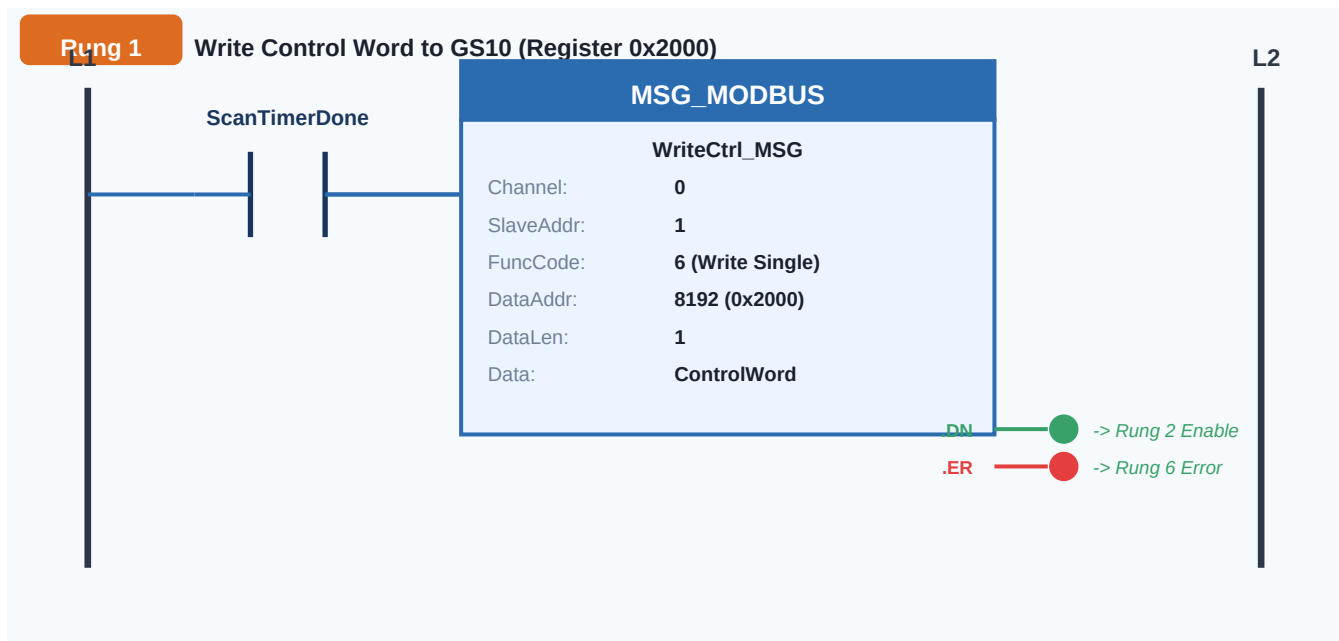
LEARNING OBJECTIVES

1. Add a MSG_MODBUS instruction to write the Control Word to the VFD.
2. Add a second MSG_MODBUS to write the Frequency Command.
3. Chain the two messages using the .DN (Done) bit.
4. Understand Modbus register addresses for the GS10.

BACKGROUND

The GS10/GS11 has special Modbus registers that control the drive. Register **8192** (hex 0x2000) is the Control Word — this is how you tell the drive to run, stop, or change direction. Register **8193** (hex 0x2001) is the Frequency Command — this is how you tell the drive what speed to run at. We use Function Code 6 (Write Single Register) to write one value at a time.

RUNG 1: Write Control Word



PROCEDURE

Step 1: Add the MSG_MODBUS Block

From the Toolbox, drag a **Block** onto Rung 1.

In the search box, type **MSG**. Select **MSG_MODBUS**.

Name the instance **WriteCtrl_MSG**. Click OK.

Step 2: Wire the Enable Input

Drag a **Normally Open Contact** to the LEFT of the MSG block.

Assign it to **ScanTimerDone**.

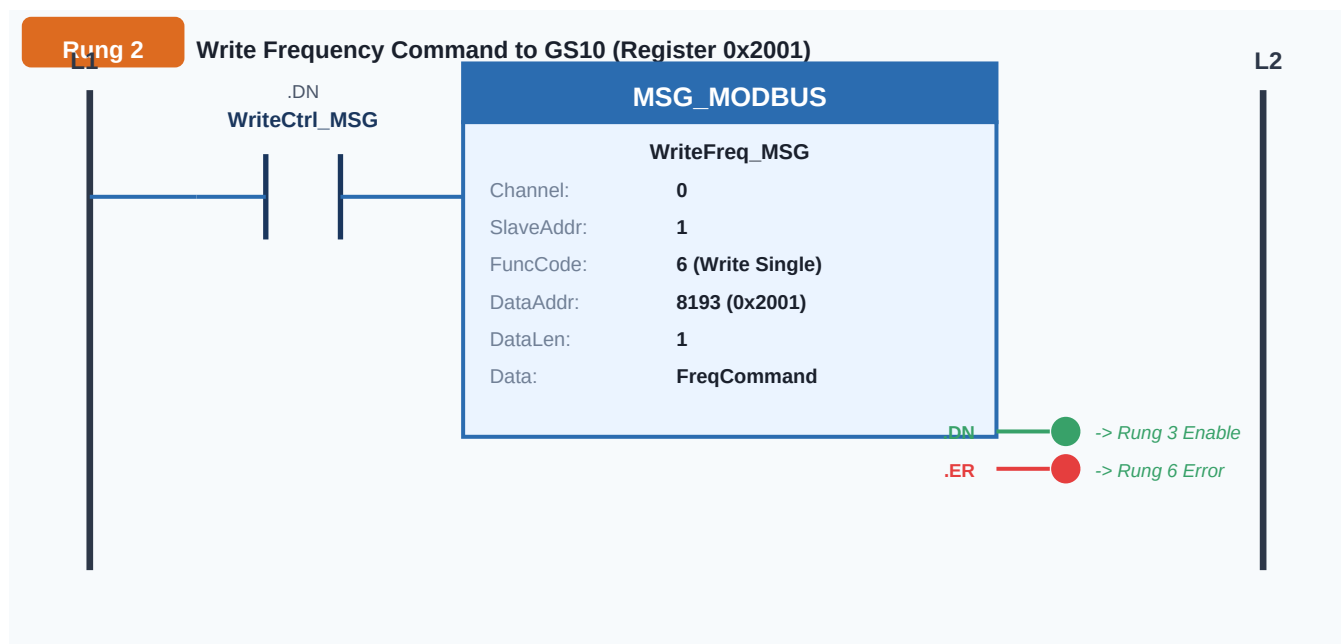
This means the write message fires once every time the timer pulses.

Step 3: Configure the MSG Parameters

Click each input pin on the block and enter these values:

Parameter	Value	What It Means
Channel	0	Use the embedded serial port
SlaveAddr	1	The drive's address (P09.00)
FunctionCode	6	Write Single Register
DataAddr	8192	Register 0x2000 = Control Word
DataLen	1	Writing just one register
Data	ControlWord	Your INT tag with the command value

RUNG 2: Write Frequency Command (Chained)



Step 4: Add Second MSG_MODBUS

Drag another Block onto Rung 2. Select MSG_MODBUS. Name it **WriteFreq_MSG**.

Step 5: Chain It — This Is the Key Step

Add a Normally Open Contact to the LEFT. Assign it to **WriteCtrl_MSG.DN**.

This is called **chaining**. Rung 2 only fires AFTER Rung 1 finishes successfully.

Without chaining, both messages would try to use the serial port at the same time and collide.

Step 6: Configure for Frequency

DataAddr: 8193 | **Data:** FreqCommand | Everything else same as Rung 1.

✓ CHECKPOINT — Verify Before Continuing

What register address is the Control Word? _____

What register address is the Frequency Command? _____

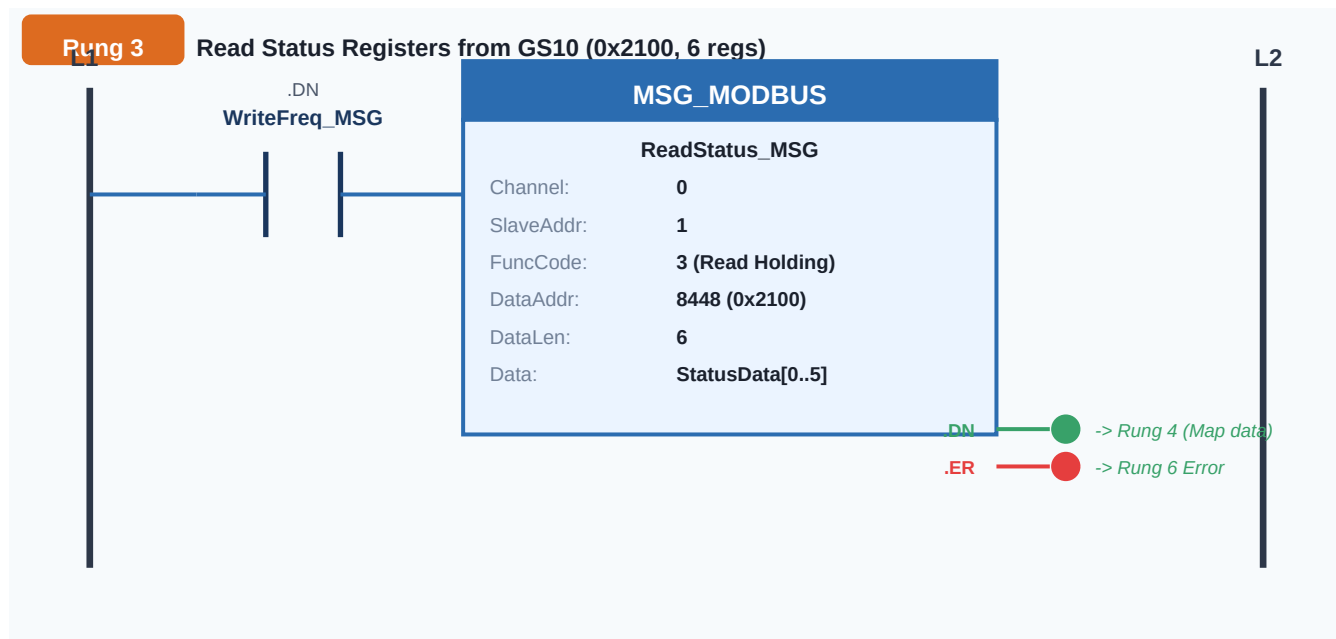
What tag did you assign to the Rung 2 enable contact? _____

Why do you chain the messages instead of triggering them both at once?

LEARNING OBJECTIVES

1. Add a MSG_MODBUS read instruction to get status back from the VFD.
2. Map the status array to named tags for fault code, frequency, and current.
3. Verify communication is working by reading live data.

RUNG 3: Read Status Registers



PROCEDURE

Step 1: Add Third MSG_MODBUS

Instance: **ReadStatus_MSG**. Enable contact: **WriteFreq_MSG.DN** (chained from Rung 2).

Step 2: Configure for READ (This Is Different!)

FunctionCode: 3 — this is Read Holding Registers, NOT 6 (write).

DataAddr: 8448 (0x2100) — this is the STATUS register, not the control register.

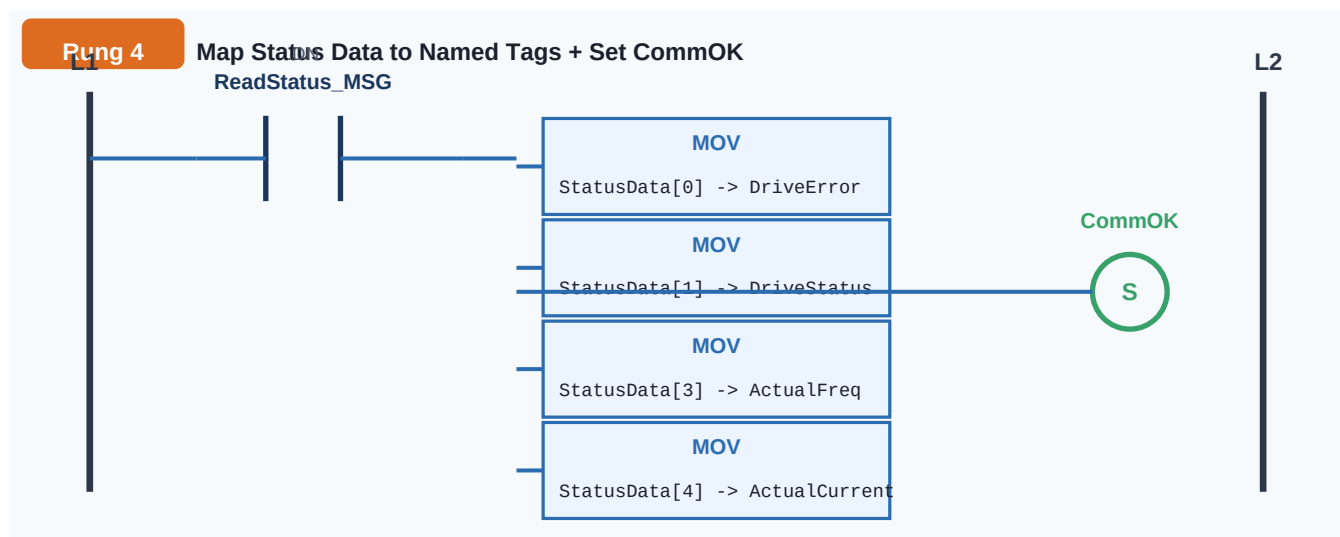
DataLen: 6 — read 6 consecutive registers.

Data: StatusData — your INT[8] array that receives the results.

What Comes Back in Your StatusData Array

Array Index	Register	What It Tells You	Example
StatusData[0]	0x2100	Fault code (0 = no fault)	0 = good
StatusData[1]	0x2101	Drive running? Direction?	Bit flags
StatusData[2]	0x2102	Commanded frequency	6000 = 60 Hz
StatusData[3]	0x2103	Actual output frequency	5985 = 59.85 Hz
StatusData[4]	0x2104	Motor current	245 = 2.45 A
StatusData[5]	0x2105	DC bus voltage	3250 = 325.0 V

RUNG 4: Map to Named Tags



Step 3: Add MOV Instructions on Rung 4

Enable contact: **ReadStatus_MSG.DN**.

Add MOV: Source=StatusData[0], Dest=DriveError.

Add MOV: Source=StatusData[3], Dest=ActualFreq.

Add SET coil: CommOK. Add RST coil: CommFault.

Step 4: Build, Download, and Go Online

Build (0 errors). Download. Go Online.

Watch the MSG .DN bits pulse in sequence — this proves communication is working!

Check that CommOK = TRUE and DriveError = 0.

✓ CHECKPOINT — Verify Before Continuing

Is CommOK = TRUE? [] Yes [] No (If No, STOP and get your instructor.)

What is DriveError reading? _____ (Should be 0)

What Function Code did you use for reading? _____

REVIEW QUESTIONS

1. What is the difference between register 8192 and register 8448?

Answer: _____

2. What Function Code writes a single register? What code reads registers?

Answer: _____

3. If StatusData[4] shows 350, what is the actual motor current in amps?

Answer: _____

LAB 7

Putting It All Together

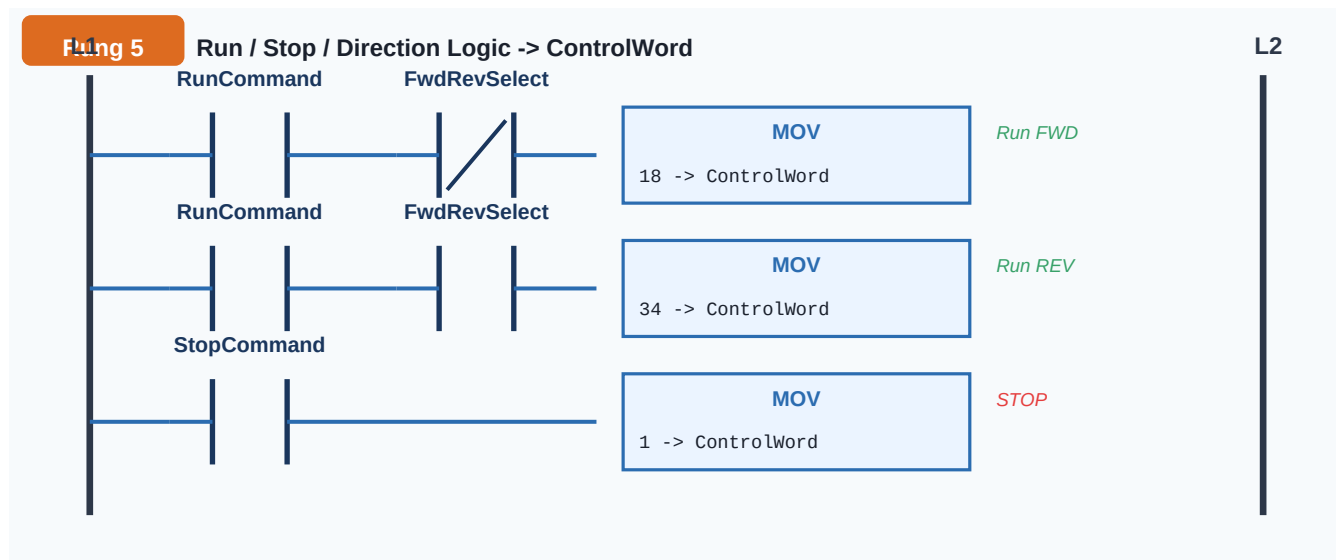
Estimated Time: 30 minutes

LEARNING OBJECTIVES

1. Build Run/Stop/Direction logic using contacts and MOV instructions.
2. Control the motor speed and direction from the PLC.
3. Read back actual speed and verify the drive responds correctly.

■ **SAFETY:** Before running the motor: (1) Motor must be disconnected from any mechanical load. (2) Verify the VFD keypad STOP button works. (3) Start at LOW speed (5 Hz) first. (4) Have your instructor verify your program before the first motor test.

RUNG 5: Run/Stop Logic



CONTROL WORD VALUES — Memorize These!

What You Want	ControlWord Value	How to Remember
Stop	1	Just the stop bit
Run Forward	18	Run (2) + Forward (16) = 18
Run Reverse	34	Run (2) + Reverse (32) = 34
Jog Forward	19	Jog+Run (3) + Forward (16) = 19

PROCEDURE

Step 1: Build the Run/Stop/Direction Rung

Follow the diagram above. Three parallel branches:

Branch 1: RunCommand (NO) + FwdRevSelect (NC) -> MOV 18 to ControlWord (Run Forward)

Branch 2: RunCommand (NO) + FwdRevSelect (NO) -> MOV 34 to ControlWord (Run Reverse)

Branch 3: StopCommand (NO) -> MOV 1 to ControlWord (Stop) — this must be LAST

Step 2: Add Speed Setpoint Copy

Add unconditional MOV: Source = SpeedSetpoint, Dest = FreqCommand.

Step 3: Build and Download. Get Instructor Approval.

Build (0 errors). Download. **DO NOT** go online yet.

Raise your hand. Have your instructor review your program before the motor test.

Instructor Approval: _____ Date: _____

Step 4: First Motor Test — SLOW SPEED

Go Online. In the Watch Window, set **SpeedSetpoint = 500** (5.00 Hz — walking speed).

Set **RunCommand = TRUE**. The motor should start spinning slowly.

Check that **ActualFreq** reads approximately 500.

Set **StopCommand = TRUE**. Motor should stop.

Step 5: Full Speed Test

Set **SpeedSetpoint = 6000** (60.00 Hz).

Set **RunCommand = TRUE**. Motor runs at full speed.

Check **ActualFreq** reads approximately 6000.

Set **StopCommand = TRUE**. Motor stops.

Step 6: Direction Test

Set **FwdRevSelect = TRUE** (reverse).

Set **RunCommand = TRUE**. Motor should spin the opposite direction.

Set **StopCommand = TRUE**.

✓ CHECKPOINT — Verify Before Continuing

Did the motor start at 5 Hz? ☐ Yes ☐ No

Did the motor stop when you set StopCommand = TRUE? ☐ Yes ☐ No

Did ActualFreq match your SpeedSetpoint? ☐ Yes ☐ Close ☐ No

Did the motor reverse direction? ☐ Yes ☐ No

FINAL REVIEW QUESTIONS

1. How many MSG_MODBUS instructions did you use in total? What does each one do?

Answer: _____

2. What would happen if you set P09.04 to 13 instead of 14?

Answer: _____

3. If you wanted to add a second VFD to the same RS-485 bus, what would you change?

Answer: _____

4. What is the purpose of chaining messages with the .DN bit?

Answer: _____

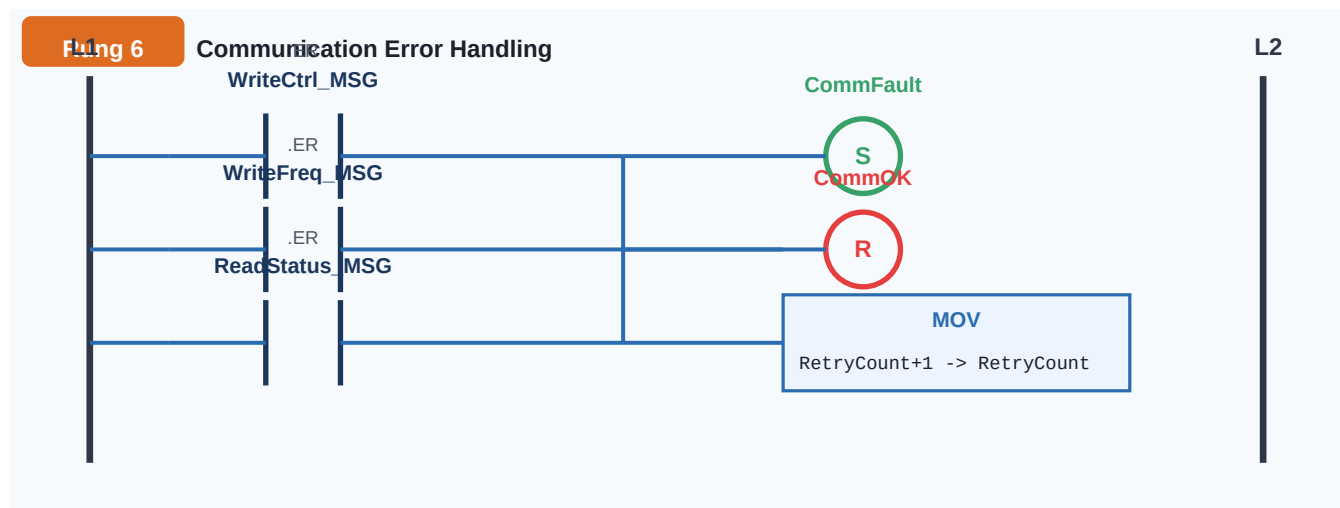
5. Why must you start motor tests at low speed (5 Hz) before going to full speed?

Answer: _____

REFERENCE: Message Chain Overview



REFERENCE: Error Handling (Rung 6)



If any MSG .ER bit fires, the error rung sets CommFault = TRUE and increments RetryCount. After 10 consecutive failures, a well-designed program would force ControlWord = 1 (Stop) to safely shut down the drive. The drive's own communication timeout (P09.02/P09.03) provides a second layer of protection.

REFERENCE: Complete Control Word Bit Map

Bits	Value	Meaning
Bits 1-0 = 00	-	No command
Bits 1-0 = 01	1	STOP
Bits 1-0 = 10	2	RUN
Bits 1-0 = 11	3	JOG + RUN

Bits	Value	Meaning
Bits 5-4 = 01	+16	Forward
Bits 5-4 = 10	+32	Reverse

To build a ControlWord value, add the numbers together. Run (2) + Forward (16) = **18**. Run (2) + Reverse (32) = **34**. Jog+Run (3) + Forward (16) = **19**.